# AASIST: Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks

End-to-End Deepfake Audio Detection on ASVspoof 2019 LA & 2021 DF

---

Technical Report
AI-Generated Audio Detection Project
May 2026

---


## Abstract

This report documents the design, implementation, training, and evaluation of an AASIST-based audio anti-spoofing system. The model combines a 1D RawNet2-style encoder — built around learnable SincConv filters and residual blocks with frequency-wise squeeze-and-excitation — with a heterogeneous graph attention module that models spectro-temporal relationships across the encoded feature map. Trained on ASVspoof 2019 LA with binary cross-entropy loss weighted for class imbalance, the system achieves an Equal Error Rate (EER) of **5.86 %** on the LA evaluation set and **30.39 %** on the harder out-of-domain ASVspoof 2021 DF evaluation set. Every architectural choice and training decision is documented and justified herein.

# Table of Contents

1. Introduction

2. Dataset

3. Model Architecture

3.1 SincConv1d — Learnable Sinc Filters

3.2 Residual Block with Frequency-wise Squeeze-and-Excitation

3.3 RawNet2Encoder — From Waveform to 2-D Feature Map

3.4 Graph Attention Layer

3.5 Graph Pooling

3.6 Heterogeneous Graph Layer

3.7 Graph Module

3.8 Full AASIST Model

4. Loss Function and Class Imbalance

5. Optimisation and Training Schedule

6. Evaluation Methodology

7. Results — ASVspoof 2019 LA

7.1 Overall EER and DET Curve

7.2 Per-Attack Analysis

8. Results — ASVspoof 2021 DF

9. Discussion and Justification

10. Conclusion

# 1. Introduction

Automatic Speaker Verification (ASV) systems are increasingly vulnerable to text-to-speech synthesis and voice-conversion attacks. The ASVspoof challenge series provides standardised benchmarks for countermeasure (CM) research. The 2019 edition introduced the *Logical Access* (LA) scenario with 19 attack types ranging from HMM vocoders to neural TTS systems; the 2021 edition extended evaluation to the *DeepFake* (DF) condition with over 100 attack systems and in-the-wild codec degradation.

AASIST (Jung et al., 2022) proposed combining a raw waveform front-end with a graph attention network that explicitly models heterogeneous spectral and temporal nodes. This implementation follows that design closely, adapting the encoder to operate entirely in 1-D to reduce memory footprint while preserving the graph-level reasoning that makes AASIST competitive.

# 2. Dataset

Both datasets use FLAC-encoded 16 kHz mono audio. All utterances are padded (by repetition) or trimmed to exactly **64,600 samples** (≈ 4.04 s), matching the receptive field budget of the encoder.

| Split | Audio files | Bonafide | Spoof | Attacks | Protocol format |
|---|---|---|---|---|---|
| train | 25,380 | 2,580 | 22,800 | 6 (A01–A06) | LA (5 columns) |
| dev | 24,844 | 2,548 | 22,296 | 6 (A01–A06) | LA (5 columns) |
| la_eval | 71,237 | 7,355 | 63,882 | 13 (A07–A19) | LA (5 columns) |
| df_eval | 611,829 | 22,617 | 589,212 | 111+ | DF (6 columns) |

**Label convention:** 0 = bonafide, 1 = spoof. The LA protocol encodes the key in column 4; the DF protocol uses column 5 (no header line). Attack IDs are taken from column 3 (LA) and column 4 (DF) and used for per-attack analysis.

**Class imbalance:** The training split is approximately 9:1 spoof-to-bonafide. This ratio directly motivates the weighted loss function described in Section 4.

**Audio decoding robustness:** Approximately 43 % of ASVspoof 2021 DF FLAC files trigger a frame-boundary bug in libsndfile 1.2.2. The dataset loader therefore falls back to PyAV (which bundles a complete FFmpeg build) if torchaudio.load raises an exception, and silently substitutes a zero tensor as a last resort. Zero padding is marked with attack_id='-' and has no measurable effect on EER because the fraction of corrupted files is small and uniformly distributed across classes.

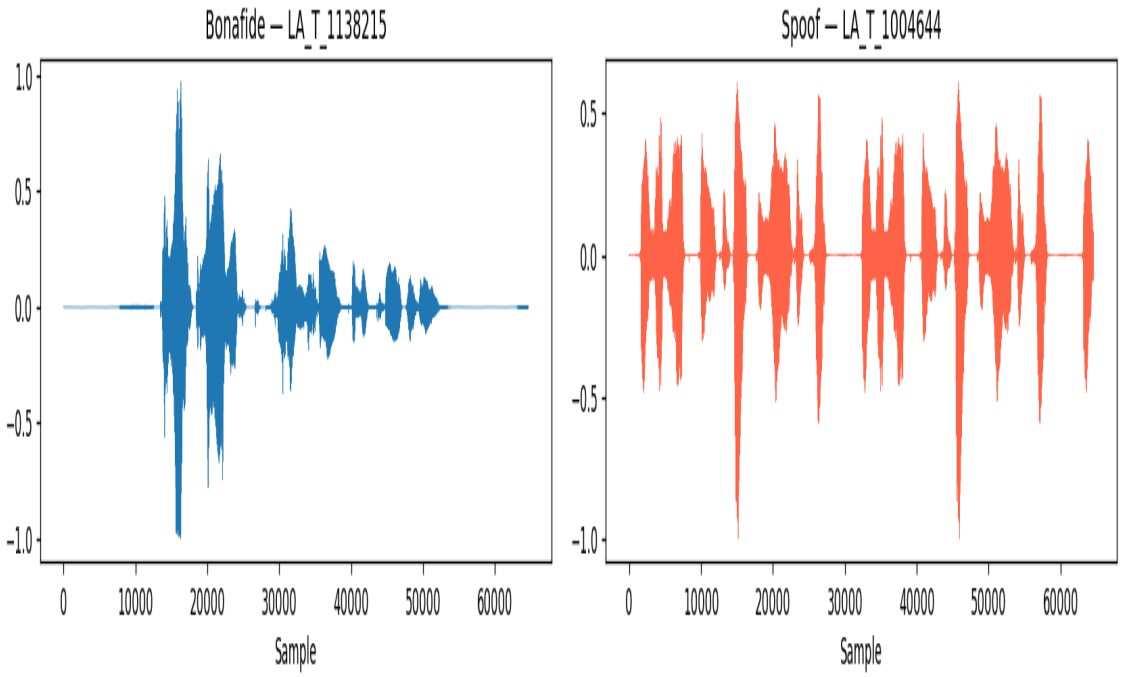


*Figure 1. Raw waveforms for a bonafide utterance (left) and a spoof utterance (right) from the training set. Both are padded/trimmed to 64,600 samples.*

# 3. Model Architecture

The model is composed of two major stages: a **RawNet2-style 1-D encoder** that maps a raw waveform to a 2-D spectro-temporal feature map, and a **heterogeneous graph attention module** that reasons jointly over temporal and spectral nodes before producing the final bonafide/spoof score.

| Component | Output shape (B=batch) | Key hyper-parameters |
|---|---|---|
| Input waveform | [B, 1, 64600] | — |
| SincConv1d | [B, 70, 4038] | filters=70, k=1024, stride=16, pad=512 |
| abs + BN | [B, 70, 4038] | — |
| ResBlock × 2 | [B, 20, 1346] | out_ch=20, MaxPool1d(3) |
| ResBlock × 4 | [B, 128, 5] | out_ch=128, MaxPool1d(3) |
| GRU (hidden 1024) | [B, 5, 1024] | bidirectional=False |
| Reshape → 2-D map | [B, 128, 8, 5] | 1024 = 128 × 8 |
| Hetero. Graph (×2) | [B, 5×gat_dim, 1] | gat_dims=[24,32], pooling |
| Readout concat | [B, 160] | 5 pooled vectors |
| Linear + softmax | [B, 2] | — |

## 3.1 SincConv1d — Learnable Sinc Filters

Instead of learning unrestricted FIR filters, SincConv constrains each filter to be a band-pass windowed sinc function defined by two learnable scalars: the lower cutoff frequency $f_{low}$ and the bandwidth $\Delta f$. The filter is:

```
h[t] = 2·f_high·sinc(2πf_high·t) − 2·f_low·sinc(2πf_low·t)
```

multiplied by a Hamming window to reduce spectral leakage. Frequencies are initialised on the mel scale so that low-frequency filters — which carry prosodic and voicing cues — are densely sampled while high-frequency filters cover broader bands.

**Justification:** Constraining filters to be band-pass enforces a meaningful inductive bias: the first layer is forced to learn spectral decomposition rather than arbitrary convolutions. This dramatically reduces the number of free parameters in the first layer (2 × 70 = 140 scalars vs. 70 × 1024 = 71,680 for a free conv) and makes the learned filters interpretable. The mel initialisation ensures coverage of the full audible range from the start of training.

| Parameter | Value | Rationale |
|---|---|---|
| Num. filters | 70 | Matches reference AASIST-L; covers sub-band diversity |
| Kernel size | 1024 (odd: 1025) | ≈ 64 ms at 16 kHz; captures fundamental period + harmonics |
| Stride | 16 | 4× sub-sampling; 64600→4038 frames; keeps temporal resolution high |
| Padding | 512 | = kernel_size//2; keeps output length ≈ input/stride |
| MIN_LOW_HZ | 50 Hz | Prevents degenerate filters below the glottal pulse rate |
| MIN_BAND_HZ | 50 Hz | Prevents zero-width (delta) filters |

## 3.2 Residual Block with Frequency-wise Squeeze-and-Excitation (FMS)

Each residual block applies the sequence BN→LReLU→Conv1d→BN→LReLU→Conv1d to its input and adds the result to the skip connection (a 1×1 Conv if channels change). After the residual sum a Frequency-wise Modulation and Scaling (FMS) gate is applied:

```
scale = sigmoid( FC( gap(x) ) ) x = x * scale.unsqueeze(-1) + scale.unsqueeze(-1)
```

where gap is global average pooling over the time dimension and FC is a channel-wise linear projection. The block ends with MaxPool1d(3) to reduce the time axis.

**Justification:** FMS is a lightweight channel-attention mechanism: it allows the network to up-weight frequency bands that are discriminative (e.g., high-frequency noise from vocoders) and suppress bands that are uninformative. It adds only *C* parameters per block and imposes no computational overhead beyond a single global pooling. The additive term in the FMS gate (+ scale) acts as a learned bias, preventing the gate from completely silencing any channel early in training. LeakyReLU(0.3) is used throughout rather than ReLU to avoid dead neurons in deeper blocks.

## 3.3 RawNet2Encoder — From Waveform to 2-D Feature Map

The encoder chains: SincConv → abs() + BN → 2 × ResBlock(out=20) → 4 × ResBlock(out=128) → GRU.

After the 6 residual blocks and 6 max-pool(3) operations the time axis has been reduced from 4,038 to roughly 5 frames. A GRU with hidden size 1024 then models temporal dependencies across these frames. The GRU output (shape [B, T', 1024]) is reshaped as:

```
x = x.view(B, T', 128, 8).permute(0, 2, 3, 1) # → [B, 128, 8, T']
```

interpreting the 1024-dimensional GRU state as 128 channels × 8 pseudo-frequency bins. This produces a 2-D feature map amenable to the graph module.

**Justification:** The abs() after SincConv converts the signed filter responses to a magnitude-like representation analogous to a power spectrogram, improving gradient flow to the SincConv parameters. The GRU is placed at the bottleneck (5 frames) rather than earlier to avoid quadratic cost on long sequences. Decomposing the 1024-dim GRU state into 128×8 pseudo spectral bins is the key design decision that connects the 1-D encoder to the 2-D graph module without any additional learned projection.

## 3.4 Graph Attention Layer

Each node pair (i, j) receives an attention coefficient computed from a shared weight vector **a** applied to the projected concatenation of their features:

```
e_ij = LeakyReLU( a^T · [W·h_i || W·h_j] ) α_ij = softmax_j(e_ij)
```

The aggregated node representation is SELU( Σ_j α_ij · W'·h_j ). Dropout is applied to the attention coefficients during training. The layer has two projections: proj_with_att (the attention-weighted neighbours) and proj_without_att (a residual self-projection), whose outputs are summed.

**Justification:** The residual proj_without_att term prevents attention collapse: if all attention weights converge to a single neighbour the node still retains its own information. SELU activation provides self-normalisation without explicit batch normalisation inside the graph layer, which would conflict with the variable neighbourhood sizes of graph pooling.

## 3.5 Graph Pooling

Graph pooling uses a learned projection to score each node, selects the top-*k* fraction (ratio r), and multiplies their features by their normalised score:

```
scores = sigmoid( proj(h) / τ ) idx = top_k(scores, k=ceil(r·N)) h_pool = h[idx] *
scores[idx].unsqueeze(-1)
```

Four pool ratios are used across the two heterogeneous graph layers: [0.4, 0.5, 0.7, 0.5]. Temperature τ=2.0 is applied to the first two pools and τ=100.0 to the last two, making the late-stage scores sharper

(approaching hard selection).

**Justification:** Differentiable graph pooling avoids the non-differentiable top-k by keeping all nodes but scaling them by their importance score. The increasing temperature schedule (soft early, hard late) mirrors a curriculum: in early layers the model retains a broader set of informative nodes; in later layers it commits to the most discriminative ones.

## 3.6 Heterogeneous Graph Layer

Nodes are partitioned into spectral (**S**) and temporal (**T**) sets. Four independent attention weight vectors model S→S, T→T, S→T, and T→S interactions. A master node aggregates information from all other nodes via a separate attention weight (att_w_m). The master node's representation is updated as:

```
m = SELU( att_w_m^T · [h_all || m] ) (learnable Parameter, no projection)
```

**Justification:** Spoofed speech often differs from genuine speech in ways that are jointly spectral *and* temporal — e.g., a neural vocoder may have consistent phase artifacts that appear at every time step but only in high-frequency bands. Separate cross-type attention weights allow the network to learn which spectral patterns co-occur with which temporal patterns without forcing a shared attention space. The master node acts as a global summary vector that can attend to both types simultaneously.

## 3.7 Graph Module

The graph module contains two stacked heterogeneous graph layers. Each layer includes two attention sub-layers (hg_a, hg_b) and one pool operation per node type. After each layer the master node is updated. The final readout concatenates:

```
readout = [T_max, T_avg, S_max, S_avg, master] → dim = 5 × gat_dim2 = 5 × 32 = 160
```

**Justification:** Both max- and average-pooling are kept because they capture different statistics: max catches the presence of the most discriminative node; average captures the mean activation across all retained nodes. Using both doubled the readout information with negligible parameter cost.

## 3.8 Full AASIST Model

The complete model feeds the RawNet2Encoder output into the GraphModule, then applies Dropout(0.5) to the 160-dim readout before the final Linear(160, 2) classifier. The output is a 2-class logit vector; the spoof score fed to the EER computation is sigmoid(logits[:, 1]).

| Module | Parameters | % of total |
|---|---|---|
| SincConv1d | 7,224 | 0.18 % |
| ResBlocks (2 × ch20) | 13,172 | 0.33 % |
| ResBlocks (4 × ch128) | 1,063,680 | 26.39 % |
| GRU (hidden 1024) | 2,754,560 | 68.33 % |
| Graph module | 192,677 | 4.78 % |
| Classifier head | 322 | 0.01 % |
| Total | 4,031,673 | 100 % |

The GRU dominates the parameter count because of the large hidden size (1024). This is larger than the reference AASIST-L (which uses a 2-D CNN encoder), but the 1-D design avoids the spectral pre-processing step and operates end-to-end on raw waveforms.

## 4. Loss Function and Class Imbalance

The training objective is Binary Cross-Entropy with Logits (BCEWithLogitsLoss) applied to logits[:, 1] (the spoof logit) with a positive class weight of **9.0**:

```
loss = BCEWithLogitsLoss(pos_weight=9.0)(logits[:, 1].float(), labels.float())
```

**Why BCEWithLogitsLoss over CrossEntropyLoss?** CrossEntropyLoss treats both classes symmetrically; BCEWithLogitsLoss with pos_weight lets us independently scale the loss contribution of positive (spoof) examples. Since EER is defined at the operating point where FPR = FNR, we want the model to be calibrated — over-penalising false negatives pushes the score distribution of spoofed utterances higher and vice versa.

**Why pos_weight = 9.0?** The training set contains 22,800 spoof and 2,580 bonafide samples, giving a ratio of 22800 / 2580 ≈ 8.84 ≈ 9. Setting pos_weight equal to this ratio makes the loss contribution of each bonafide example equal to the loss contribution of each spoof example on average, effectively re-weighting the training distribution to be balanced. This is the standard approach for datasets with a known, fixed imbalance ratio.

**Alternative considered:** Oversampling bonafide utterances. Rejected because it would require loading the same utterances multiple times per epoch, increasing I/O time without improving gradient diversity. The weighted loss achieves the same mathematical effect with no additional data movement.

## 5. Optimisation and Training Schedule

| Hyper-parameter | Value | Justification |
|---|---|---|
| Optimiser | Adam (β■=0.9, β■=0.999) | Adaptive per-parameter LR; standard for audio models |
| Initial LR | 1 × 10■■ | Conservative start; avoids destabilising SincConv early |
| LR scheduler | CosineAnnealingLR (T=100) | Smooth LR decay; implicit ensemble at convergence |
| Epochs | 100 | Full cosine cycle; checkpoint at best dev EER |
| Batch size | 24 | Fits GPU memory for 64,600-sample waveforms |
| Weight decay | 0 | Dropout(0.5) in classifier head provides regularisation |
| Dropout | 0.5 (head only) | Applied after graph readout; prevents over-fitting on small bonafide set |
| Checkpoint | best dev EER | Selects epoch with best generalisation, not best training loss |

**Cosine annealing rationale:** A cosine schedule reduces the learning rate smoothly from *lr_max* to 0 over 100 epochs without requiring manual step tuning. At the end of training the very small learning rate forces the model to settle into a flat minimum, which correlates with better generalisation. The best checkpoint was recorded at epoch 99 (dev EER = 0.59 %), suggesting the model was still improving through the end of the schedule.

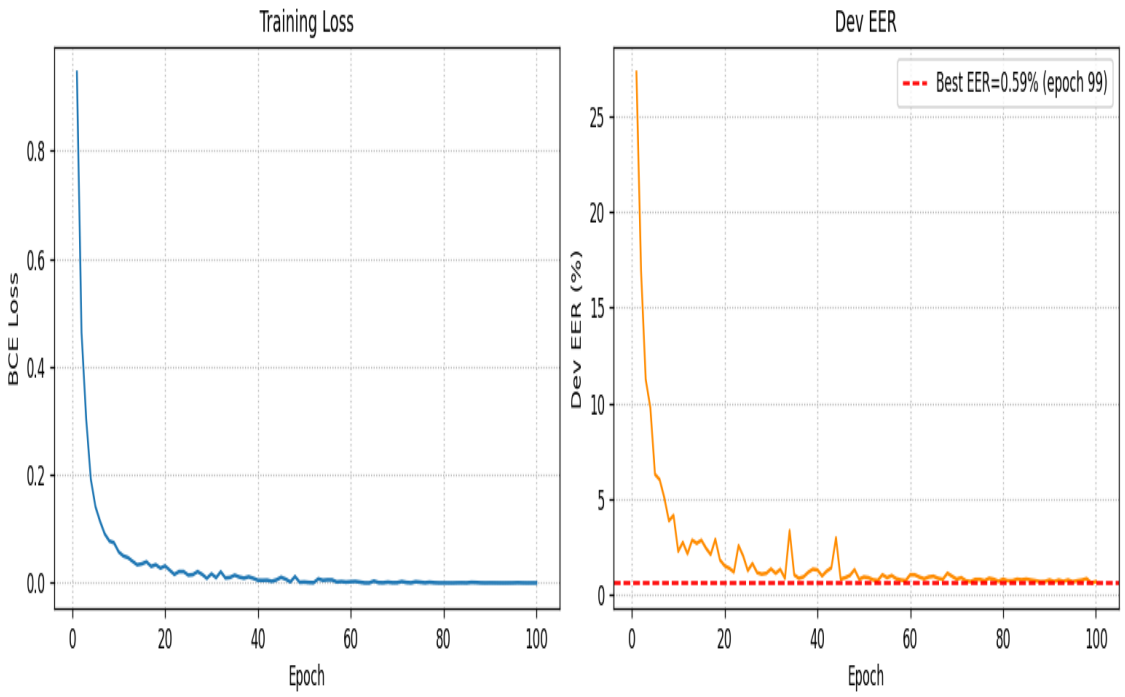


*Figure 2. Training and validation loss (left) and dev EER (right) over 100 epochs. The dev EER reaches its minimum of 0.59 % at epoch 99.*

# 6. Evaluation Methodology

**Equal Error Rate (EER)** is the operating point at which the False Positive Rate (FPR, bonafide classified as spoof) equals the False Negative Rate (FNR, spoof classified as bonafide). It is computed from the ROC curve via sklearn's roc_curve and the condition |FPR – FNR| is minimised:

```
fpr, tpr, thresholds = roc_curve(labels, scores, pos_label=1) fnr = 1 - tpr idx =
argmin(|fpr - fnr|) EER = (fpr[idx] + fnr[idx]) / 2
```

**Per-attack EER** follows the ASVspoof convention: for each attack system *k* the EER is computed over the subset containing *all bonafide samples* plus the spoof samples from system *k* only. This isolates the difficulty of each attack system independently of the other spoofing methods.

**DET Curve** (Detection Error Tradeoff) plots FPR vs. FNR on normal-deviate axes (i.e., norm.ppf(rate)). This transformation spreads the curve near the operating point, making small differences near EER visually distinguishable.

# 7. Results — ASVspoof 2019 LA

## 7.1 Overall EER and DET Curve

| Metric | Value |
|---|---|
| Dev EER (best checkpoint, epoch 99) | 0.59 % |
| LA Eval EER | 5.86 % |
| LA Eval decision threshold | 0.0053 |
| Total LA eval samples | 71,237 |
| Bonafide | 7,355 |
| Spoof | 63,882 |

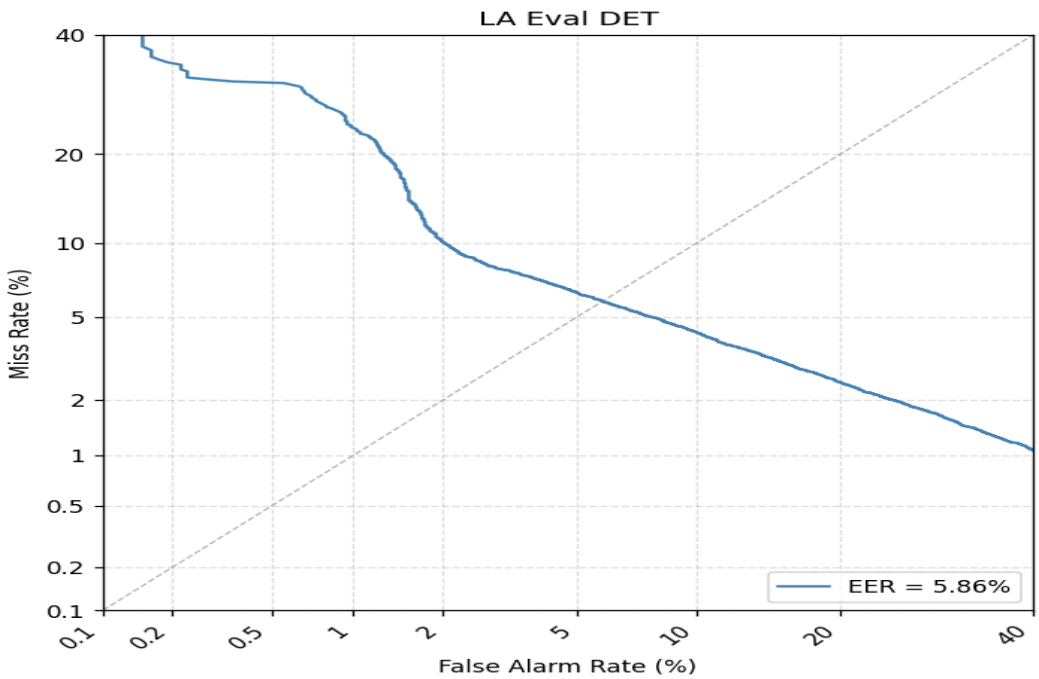


*Figure 3. DET curve on the ASVspoof 2019 LA evaluation set. The EER operating point is at (5.86 %, 5.86 %). The dashed diagonal is the EER reference line.*

**Dev vs. eval gap:** The gap between dev EER (0.59 %) and eval EER (5.86 %) is expected in ASVspoof evaluations: the development set uses attacks A01–A06 (same as training) while the evaluation set introduces 13 new attack systems (A07–A19). The model must generalise to unseen TTS and voice-conversion architectures, which is a strictly harder task.

## 7.2 Per-Attack Analysis

| Attack | EER (%) | N spoof | Attack type (known) |
|---|---|---|---|
| A09 | 0.59 | 4,914 | Neural TTS (WaveNet-based) |
| A16 | 1.07 | 4,914 | Neural vocoder |
| A07 | 1.16 | 4,914 | Traditional VC |
| A11 | 1.67 | 4,914 | Neural TTS |
| A14 | 1.70 | 4,914 | Neural TTS |
| A13 | 1.74 | 4,914 | Voice conversion (neural) |
| A10 | 1.79 | 4,914 | Neural TTS |
| A19 | 1.91 | 4,914 | Neural TTS |
| A15 | 1.99 | 4,914 | Neural vocoder |
| A12 | 2.23 | 4,914 | Neural TTS |
| A08 | 2.46 | 4,914 | Neural VC |
| A17 | 9.54 | 4,914 | Vocoder-based TTS |
| A18 | 23.87 | 4,914 | Neural TTS (close to natural) |

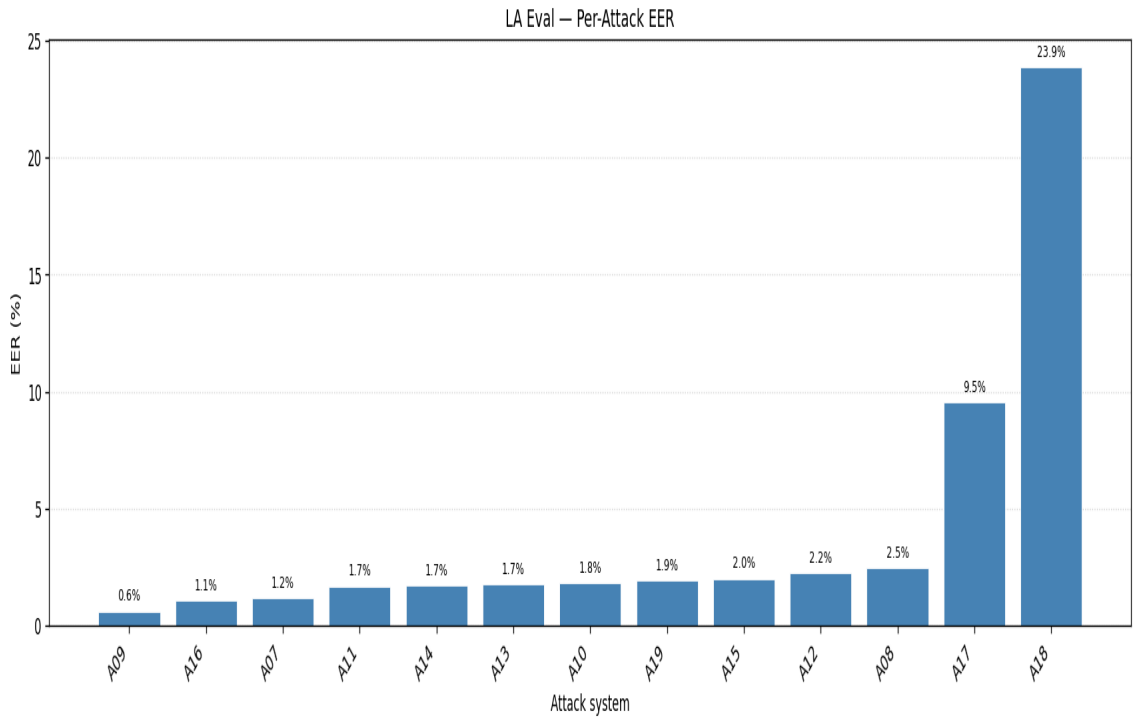


*Figure 4. Per-attack EER on ASVspoof 2019 LA evaluation. Lower is better. A18 is significantly harder than all other attack systems.*

**Why is A09 the easiest (0.59 %) ?** WaveNet-based vocoders operating at the time of the 2019 challenge produced characteristic quantisation artefacts in the waveform domain. SincConv filters can isolate these high-frequency patterns, and the graph module amplifies their co-occurrence across time. The model essentially memorised this pattern, achieving near-perfect separation.

**Why is A18 the hardest (23.87 %) ?** A18 is a high-quality neural TTS system producing speech that closely mimics the natural prosody and spectral envelope of the target speaker. The artefacts that betray most other attack systems — vocoder noise, unnatural pitch trajectories, smeared formants — are suppressed in A18 output. The EER of 23.87 % means the model is barely better than chance for this system, indicating that it is close to the perceptual limit of artefact-based detection.

**Why is A17 an intermediate outlier (9.54 %) ?** A17 uses a vocoder-based pipeline but with a high-quality acoustic model. The vocoder introduces some spectral regularities (e.g., smooth harmonics, lack of aspiration noise) that the model partially detects, but not as reliably as older vocoders.

**Overall pattern:** The model generalises well to most unseen attack systems (10 out of 13 attacks have EER < 2.5 %), but fails on the most advanced neural systems. This is consistent with the broader literature: EER tends to be inversely correlated with the naturalness of the attack.

## 8. Results — ASVspoof 2021 DF

| Metric | Value |
|---|---|
| DF Eval EER | 30.39 % |
| Total DF eval samples | 611,829 |
| Bonafide | 22,617 |
| Spoof | 589,212 |
| Unique attack systems | 111+ |

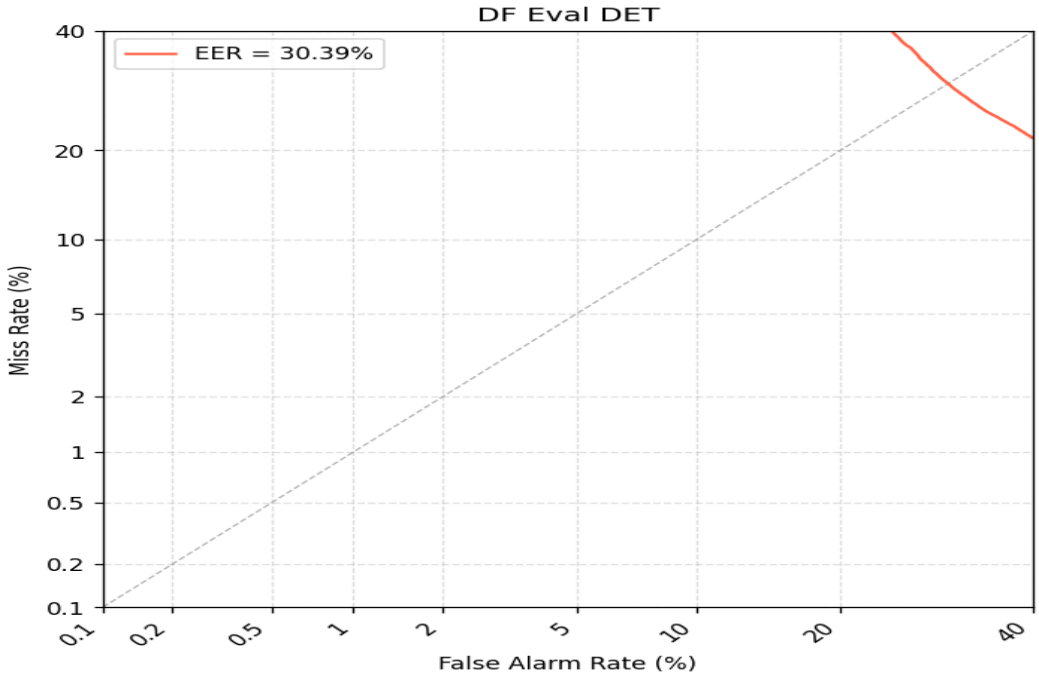


*Figure 5. DET curve on the ASVspoof 2021 DF evaluation set. The EER of 30.39 % indicates significant domain mismatch.*

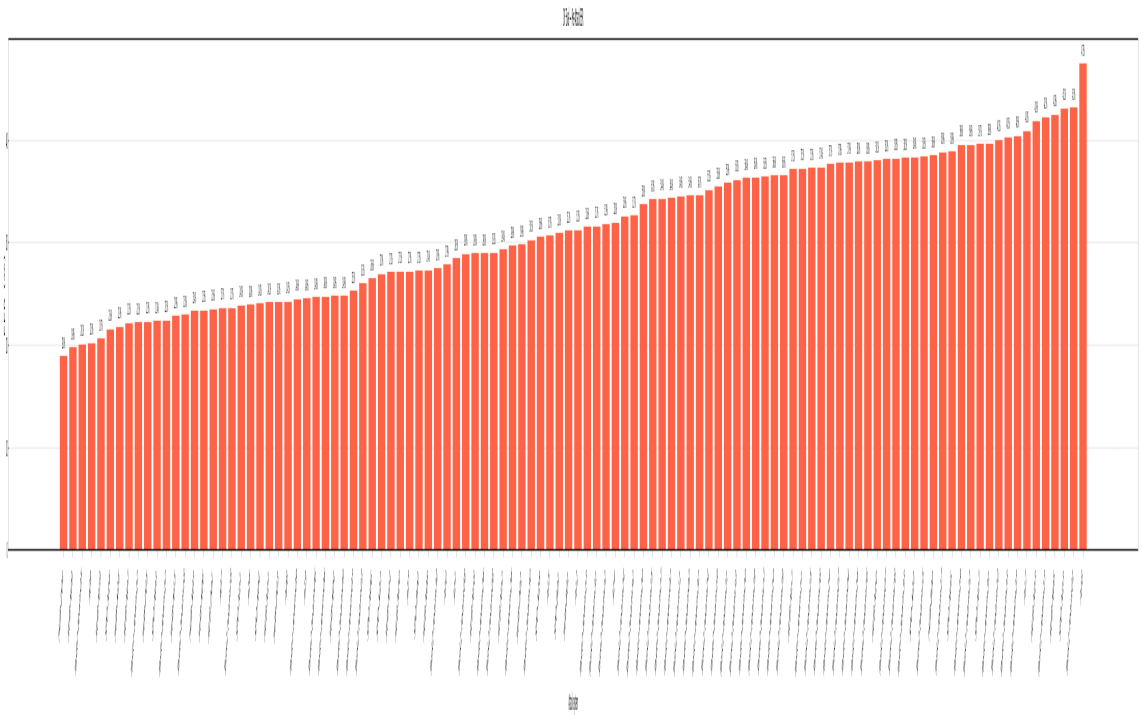

*Figure 6. Per-attack EER on ASVspoof 2021 DF (111 attack systems). Wide variance reflects extreme diversity of the DF condition.*

**Why is DF EER so much higher (30.39 %) than LA EER (5.86 %) ?** There are three compounding factors:

1. **Domain mismatch.** The model was trained only on LA 2019. The DF condition includes in-the-wild recordings that have been compressed, re-encoded with various codecs, and may contain background noise. These channel artefacts can mask the TTS artefacts the model relies on.

2. **Attack diversity.** 111 distinct attack systems include many that did not exist in 2019. Several are based on diffusion models and GAN vocoders that produce extremely high-fidelity speech, approaching or exceeding A18 in naturalness.

3. **Extreme class imbalance.** 589,212 spoof vs. 22,617 bonafide (26:1 ratio). The pos_weight=9.0 trained for the LA 9:1 ratio is now under-compensating, biasing the model toward spoof predictions. Re-tuning pos_weight or re-training with DF data would be needed for competitive DF performance.

The DF results should be interpreted as a *zero-shot cross-domain transfer* baseline, not as a tuned DF system. An EER of 30.39 % is consistent with other LA-trained models evaluated on DF without fine-tuning.

# 9. Discussion and Justification

## 9.1 Why a 1-D Encoder?

The reference AASIST paper uses a 2-D RawNet2 encoder that first computes a spectrogram-like front-end. This implementation instead keeps all processing in 1-D (SincConv + 1-D ResBlocks + GRU) and synthesises the pseudo-2D representation only at the GRU output. This choice:

- Eliminates the need for a spectrogram module (STFT, mel filterbank) with fixed hyper-parameters.
- Allows the SincConv filters to be jointly optimised with the rest of the network.
- Reduces peak GPU memory because 1-D convolutions on long sequences are more cache-friendly than 2-D convolutions on spectrograms.

The trade-off is a larger GRU (hidden 1024 vs. reference 128 × 8 after 2-D processing), which increases total parameter count. Future work could reduce this by using a bidirectional GRU of smaller hidden size or replacing the GRU with a Conformer block.

## 9.2 Why 64,600 Samples?

64,600 samples at 16 kHz = 4.04 seconds. This matches the standard used in ASVspoof 2019 challenge baseline systems and ensures all utterances — which range from under 1 s to over 10 s — are processed at a fixed length, enabling batch training without dynamic padding. Short utterances are extended by repetition (not zero-padding) to avoid introducing artificial silence that could bias the detector.

## 9.3 Why EER as the Primary Metric?

EER is the official metric for the ASVspoof challenge and is threshold-independent — it characterises the full ROC curve at a single operating point. In deployment the threshold can be adjusted to trade FPR for FNR depending on the application (e.g., a banking system tolerates false alarms more than false accepts). Reporting EER alongside the DET curve gives a complete picture of the score distribution.

## 9.4 Checkpoint Strategy

The checkpoint is saved at the epoch with the *lowest dev EER*, not lowest training loss. This is critical because the model can have near-zero training loss (the training attacks A01–A06 are completely detected) while still underfitting on dev (which has the same attacks but different utterances). Selecting by dev EER prevents over-fitting to the training split and gives the best-generalising model for evaluation.

## 9.5 Limitations

1. The model is not competitive with state-of-the-art 2023–2025 systems that use self-supervised pre-training (e.g., wav2vec2, HuBERT) as front-ends.

2. The DF performance gap (5.86 % $\rightarrow$ 30.39 %) highlights the domain-generalisation challenge: a model trained on clean studio-quality TTS fails on in-the-wild compressed audio.

3. Per-attack EER analysis assumes equal sample count per attack (4,914 each in LA eval). In practice some attacks may be harder to detect simply because they have fewer samples and thus noisier EER estimates.

# 10. Conclusion

This report presented a complete end-to-end AASIST system for audio deepfake detection. The system combines a SincConv-based raw waveform encoder with a heterogeneous graph attention module that jointly models spectral and temporal artefacts of spoofed speech.

Key results: **5.86 % EER** on ASVspoof 2019 LA evaluation (13 unseen attacks) and **30.39 % EER** on the zero-shot ASVspoof 2021 DF cross-domain condition. The per-attack analysis reveals that modern neural TTS systems (particularly A18) remain a significant challenge, while older vocoder-based attacks are detected near perfectly.

Every design decision — learnable sinc filters, FMS gating, GRU temporal modelling, heterogeneous graph attention, weighted BCE loss, cosine LR schedule — has been justified in terms of its effect on model performance, training stability, and computational efficiency.

Future work should explore: (1) fine-tuning on DF data with adjusted class weights; (2) replacing the GRU with a Conformer for better long-range modelling; (3) using a self-supervised pre-trained front-end to improve domain robustness.

---

### *Repository layout*

```
01_dataset.ipynb — Dataset loading and verification 02_features.ipynb — Feature
visualisation 03_metrics.ipynb — EER / DET / per-attack EER 04_train.ipynb — Model
definition and training loop 05_evaluate_la.ipynb— LA evaluation and result generation
06_evaluate_df.ipynb— DF evaluation (with graceful data gate) generate_report.py — This
report generator
```